

Session Reliability Engineering

Media Framework, SDK and Connectivity Debugging Project

The resume line this document explains

Root-caused and resolved 60+ complex issues spanning the media framework, SDK, and connectivity layer, raising session reliability to 99.9% by adding packet captures, trace logging, and crash symbolication with LLDB and Instruments.

A plain-language guide: summary, components, workflow and architecture

Contents

1. Overall Summary (in simple terms)	3
1.1 Key numbers explained	3
1.2 Key terms glossary	4
2. Components Explained (end to end)	6
2.1 Application Layer	6
2.2 SDK Layer	7
2.3 Media Framework	9
2.4 Connectivity Layer	13
2.5 Network and Servers	16
2.6 Observability and Debugging Toolkit (added in this project)	16
3. Workflow Explained (end to end)	21
3.1 Runtime workflow: life of one session	21
3.2 Debugging workflow: how each of the 60+ issues was solved	22
3.3 Example issues walked through end to end	24
4. Architecture Diagram	25
4.1 How to read the diagram	26
4.2 Layer-by-layer explanation	26
4.3 Arrow-by-arrow explanation	27
4.4 Observability connections (red elements)	28
4.5 Full end-to-end trace through the diagram	28
5. Results and Impact	30
5.1 How to explain this in an interview (30 seconds)	30

(If page numbers do not appear, right-click the contents area in Word and choose "Update Field".)

1. Overall Summary (in simple terms)

Imagine an app that lets two people talk on a video call, or lets someone watch a live stream. Each of these conversations is called a **session**. A session is “reliable” when it starts properly, keeps working the whole time, and ends cleanly without freezing, losing sound, dropping the call, or crashing the app.

Before this project, sessions sometimes failed. Calls dropped, video froze, sound came through only one way, or the app crashed. The hard part was that nobody could easily see **why**, because the software did not leave enough clues behind.

This project did two big things:

1. **Added “eyes and ears” to the system (observability).** Detailed trace logs (a diary of what the code is doing), packet captures (recordings of the data travelling over the network), and crash symbolication (turning unreadable crash reports into exact lines of code). Apple’s professional tools, **LLDB** (a debugger that pauses the app to inspect it) and **Instruments** (a profiler that measures CPU, memory and timing), were used to dig deeper.
2. **Used those clues to find and fix the real cause of 60+ difficult problems** across three parts of the system: the **media framework** (handles audio and video), the **SDK** (the toolkit app developers use), and the **connectivity layer** (handles the network connection).

The result: **99.9% of sessions now succeed**. In plain numbers, out of every 1,000 calls or streams, at least 999 work correctly from start to finish.

One-sentence version

I gave the calling/streaming system the instruments a doctor would use (X-rays, heart monitors, lab tests), used them to diagnose and cure more than 60 hard “illnesses” in the audio/video, toolkit and network parts of the app, and as a result almost every session (99.9%) now works perfectly.

1.1 Key numbers explained

Number	What it means in simple terms
60+ issues	More than sixty separate bugs, each needing investigation to find its true source rather than a quick patch over the symptom.
3 layers	Problems were spread across three different parts of the system (media, SDK, connectivity), so the work required understanding the whole pipeline end to end.
99.9% reliability	Out of every 1,000 sessions, 999 or more start, run and finish correctly. Failure rate went down to 1 in 1,000 or fewer.

1.2 Key terms glossary

Term	Simple meaning
Session	One complete call or stream, from pressing “start” to pressing “end”.
Root cause	The real, underlying reason a problem happens, not just the visible symptom.
Media framework	The part of the software that records, compresses, sends, receives, decodes and plays audio and video.
SDK (Software Development Kit)	A ready-made toolkit that app developers plug into their app to add calling/streaming without building it from scratch.
Connectivity layer	The part that finds a network path to the other device and keeps the data flowing over it.
Packet	A small envelope of data sent over the network. Audio and video are split into thousands of packets per second.
Packet capture	A recording of every packet sent and received, like a CCTV recording of the network.
Trace logging	Time-stamped notes the code writes about what it is doing, like a ship’s logbook.
Crash symbolication	Translating a crash report full of memory addresses (e.g. 0x1a2b3c) into readable function names and line numbers.
dSYM file	A “dictionary” file created when the app is built, used to translate addresses into code names during symbolication.
LLDB	Apple’s debugger. It can pause a running app, look inside variables and step through code line by line.
Instruments	Apple’s profiling tool. It measures CPU use, memory, leaks, threads, energy and network activity over time.
Codec (H.264, Opus)	A method of shrinking audio/video so it fits through the network, then expanding it again on the other side.
Jitter	Packets arriving at uneven times. A jitter buffer smooths this out.

Term	Simple meaning
NAT / Firewall	Home and office routers that hide devices; they often block direct connections.
ICE / STUN / TURN	Standard techniques for getting through routers: STUN discovers your public address, TURN relays traffic when direct paths fail, ICE tries all options and picks the best.
RTP / SRTP	The standard formats for sending real-time audio/video packets; SRTP is the encrypted version.
Signaling	The “setup conversation” before and during a call (who is calling, which codecs, which network addresses).

2. Components Explained (end to end)

The system is made of six groups of components. Groups 1 to 5 are the product itself (what users experience). Group 6 is the observability and debugging toolkit that this project added and used. Each component below follows the same pattern: what it is, an analogy, how it works step by step, the problems found there, and how they were fixed.

Note

The resume line names the three layers and the tools. The specific sub-components, bug examples and fixes below describe the standard design of real-time audio/video systems on Apple platforms and are representative of the kinds of issues such a project resolves. Adjust names and examples to match your exact codebase.

2.1 Application Layer

App UI (call / stream screen)

Aspect	Simple explanation
What it is	The screens and buttons the user sees: start call, end call, mute, switch camera, video tiles.
Everyday analogy	The dashboard of a car: the driver only touches the steering wheel and pedals, not the engine.
Where it sits	Top of the stack. It talks only to the SDK's public API.

How it works, step by step:

- User taps "Start call".
- The app asks iOS for camera and microphone permission if not already granted.
- The app calls the SDK (for example, `startSession(roomID)`).
- The app listens for SDK callbacks (connected, reconnecting, ended, error) and updates the screen.
- When the user taps "End", the app calls the SDK to stop the session and clean up.

Typical problems found here:

- UI updated from a background thread, causing glitches or crashes.
- App not handling an error callback, so the screen stayed stuck on "Connecting..."
- App going to the background (phone call, lock screen) without telling the SDK.

How it was debugged and fixed:

- Trace logs showed the exact order of API calls and callbacks, revealing missing handlers.
- Main Thread Checker and LLDB breakpoints caught UI updates on the wrong thread.
- Fixes: dispatch UI work to the main thread, handle every error state, react to app lifecycle events.

2.2 SDK Layer

Public API

Aspect	Simple explanation
What it is	The set of functions app developers call: start, stop, mute, unmute, switch camera, set quality.
Everyday analogy	A restaurant menu: customers order from the menu, they do not walk into the kitchen.
Where it sits	Entry point of the SDK, directly below the App UI.

How it works, step by step:

- Receives a request from the app (e.g. start session).
- Validates inputs (is the room ID valid? are permissions granted?).
- Hands the request to the Session Manager.
- Returns immediately; results come back later through callbacks.

Typical problems found here:

- Calling start twice quickly created two overlapping sessions.
- Calling stop while still connecting left resources half-open.
- Unclear error codes made it impossible for apps to react correctly.

How it was debugged and fixed:

- Trace logs with a unique session ID on every line showed duplicate calls.
- Fixes: make calls idempotent (calling twice is safe), queue requests in order, return clear, specific error codes.

Session Manager and State Machine

Aspect	Simple explanation
What it is	The “brain” that tracks what state a session is in and decides what happens next.
Everyday analogy	A traffic light controller: it only allows legal changes (red → green → yellow → red), never jumps randomly.
Where it sits	Heart of the SDK. It coordinates both the media framework and the connectivity layer.

How it works, step by step:

- States: Idle → Connecting → Connected → Reconnecting → Disconnecting → Ended (and Failed).
- On start: moves to Connecting, tells connectivity to set up signaling, and tells media to start capture.
- When the network path is ready and media flows: moves to Connected.
- If the network drops: moves to Reconnecting and asks connectivity to recover.
- On stop or fatal error: tears everything down in a fixed, safe order and moves to Ended or Failed.
- Every state change is written to the trace log with a timestamp and reason.

Typical problems found here:

- Race conditions: two threads changing state at the same time, causing an illegal state (e.g. Connected after Ended).
- Missing timeouts: a session stuck in Connecting forever.
- Teardown in the wrong order: media still writing to a network socket that was already closed, causing a crash.

How it was debugged and fixed:

- Signposts in trace logs plus Instruments’ timeline showed overlapping state transitions on different threads.
- LLDB watchpoints on the state variable caught exactly who changed it.
- Fixes: serialize all state changes on one queue, add a timeout for each state, enforce a strict teardown order.

Callbacks and Error Reporting

Aspect	Simple explanation
What it is	The way the SDK tells the app what happened: connected, quality changed, reconnecting, ended, error.
Everyday analogy	Text notifications from a delivery service: “out for delivery”, “delivered”, “delayed”.
Where it sits	Bottom of the SDK, sending events back up to the App UI.

How it works, step by step:

- Session Manager emits an event whenever something important changes.
- The callback layer converts it into a public event with a clear error code and message.
- Events are delivered on a predictable thread (usually the main thread) so the app can safely update the UI.
- Each event is also logged, so the logs show exactly what the app was told and when.

Typical problems found here:

- Callbacks fired after the app had released its object, causing crashes.
- Events delivered out of order (“ended” before “connected”).
- Generic “unknown error” hiding the real cause.

How it was debugged and fixed:

- Symbolicated crash reports pointed directly to callbacks touching freed memory.
- Fixes: weak references to app delegates, ordered serial delivery, a detailed error-code catalogue.

2.3 Media Framework

The media framework is a production line for audio and video. On the sending side it records, compresses and packages. On the receiving side it smooths, decompresses and plays.

Capture (camera and microphone)

Aspect	Simple explanation
What it is	Collects raw video frames from the camera and raw audio samples from the microphone.
Everyday analogy	The camera crew and sound recorder on a film set.

Aspect	Simple explanation
Where it sits	First stage of the sending pipeline (uses AVFoundation / AVCaptureSession and Audio Units).

How it works, step by step:

- Configures the camera (resolution, frame rate) and the audio session (category, sample rate).
- Receives roughly 30 video frames per second and continuous audio buffers.
- Applies basic processing: echo cancellation and noise suppression for audio, rotation for video.
- Passes each frame or audio buffer to the encoder with a timestamp.

Typical problems found here:

- Audio route changes (plugging in headphones, connecting Bluetooth) stopping audio entirely.
- Interruptions (an incoming phone call) not resumed properly, causing one-way audio.
- Camera not released, so other apps could not use it afterwards.

How it was debugged and fixed:

- Trace logs around audio-session notifications revealed route changes and interruptions that were not handled.
- Fixes: handle every interruption/route-change event, restart the audio unit safely, always release the camera on teardown.

Encoder (H.264 video / Opus audio)

Aspect	Simple explanation
What it is	Compresses raw audio and video so they are small enough to send over the internet.
Everyday analogy	Vacuum-packing clothes so they fit in a suitcase.
Where it sits	Between Capture and Transport on the sending side (uses VideoToolbox for video).

How it works, step by step:

- Receives raw frames and audio buffers with timestamps.
- Compresses video into keyframes (full pictures) and delta frames (only what changed).
- Compresses audio into small Opus frames (about 20 ms each).
- Adjusts bitrate up or down when told the network is good or poor.

- Hands compressed data to Transport for packetization.

Typical problems found here:

- Hardware encoder session invalidated when the app went to the background, leading to frozen video on return.
- Memory growth from encoded frames that were never released.
- Bitrate not lowered on poor networks, flooding the connection.

How it was debugged and fixed:

- Instruments (Allocations and Leaks) showed steady memory growth tied to the encoder callback.
- Fixes: recreate the encoder session when invalidated, release buffers correctly, connect bandwidth estimates to bitrate control.

Jitter Buffer

Aspect	Simple explanation
What it is	A short waiting area for incoming packets that puts them back in order and releases them at a steady pace.
Everyday analogy	A waiting room at a clinic: patients arrive at random times, but are called in one by one at a steady rhythm.
Where it sits	First stage of the receiving pipeline, right after Transport.

How it works, step by step:

- Receives packets that may arrive late, early, out of order or duplicated.
- Uses sequence numbers to reorder them.
- Holds them briefly (typically 20–200 ms, adjusted automatically).
- Requests a resend (NACK) or a fresh keyframe if important packets are missing.
- Releases complete frames to the Decoder at a smooth pace.

Typical problems found here:

- Buffer too small: choppy audio on slightly unstable networks.
- Buffer growing without limit: increasing delay until the call felt like a walkie-talkie.
- Sequence-number wrap-around (after 65,535) treated as very old packets, freezing video.

How it was debugged and fixed:

- Packet captures in Wireshark showed packets actually arrived; logs showed the buffer discarded them, pointing to the wrap-around bug.

- Fixes: correct wrap-around math, cap maximum delay, tune adaptive sizing.

Decoder

Aspect	Simple explanation
What it is	Expands compressed audio and video back into pictures and sound.
Everyday analogy	Unpacking the vacuum-packed suitcase at the destination.
Where it sits	Between the Jitter Buffer and the Renderer.

How it works, step by step:

- Receives complete compressed frames from the jitter buffer.
- Starts from a keyframe, then applies delta frames to rebuild each picture.
- Decodes Opus audio frames into playable samples; conceals small gaps if a packet is lost.
- Passes decoded frames to the Renderer with timestamps for lip-sync.

Typical problems found here:

- Crashes when a malformed or truncated frame arrived.
- Green or corrupted video after packet loss because decoding continued without a keyframe.
- Decoder used after it had been destroyed during teardown.

How it was debugged and fixed:

- Crash symbolication turned raw addresses into the exact decoder function and line.
- LLDB reproduced the crash with a saved bad packet from the capture.
- Fixes: validate frame data before decoding, request a keyframe after loss, protect decoder lifetime during teardown.

Renderer (screen and speaker)

Aspect	Simple explanation
What it is	Shows the video on the screen and plays the audio through the speaker or headphones.
Everyday analogy	The cinema projector and loudspeakers.
Where it sits	Last stage of the receiving pipeline.

How it works, step by step:

- Receives decoded frames and audio samples.
- Uses timestamps to keep lips and voice in sync.
- Draws video using Metal or a display layer on the main/render thread.
- Feeds audio to the playback audio unit at a steady rate.

Typical problems found here:

- Audio and video drifting out of sync over long calls.
- Dropped frames and stutter because rendering blocked the main thread.

How it was debugged and fixed:

- Instruments (Time Profiler and Core Animation) showed heavy work on the main thread.
- Fixes: move work off the main thread, use the audio clock as the master for A/V sync.

2.4 Connectivity Layer

Signaling (TLS / WebSocket)

Aspect	Simple explanation
What it is	The control channel used to set up, change and end a session. It carries messages, not audio/video.
Everyday analogy	Phoning a friend to agree on the time and place to meet, before you actually meet.
Where it sits	Talks to the Signaling Server; directed by the Session Manager.

How it works, step by step:

- Opens a secure WebSocket connection to the signaling server.
- Sends an “offer” describing supported codecs and network candidates.
- Receives an “answer” from the other side.
- Exchanges network candidates (possible addresses) with the other side.
- Sends keep-alive pings so the server knows the client is still there.
- Sends a “bye” message when the session ends.

Typical problems found here:

- WebSocket silently dropped by mobile networks without either side noticing.
- Messages lost during reconnect, leaving both sides in different states.
- TLS handshake failures on certain corporate networks.

How it was debugged and fixed:

- Packet captures showed missing keep-alive replies and TCP resets.
- Fixes: heartbeat with timeout detection, message acknowledgement and resend on reconnect, clearer TLS error reporting.

ICE / STUN / TURN (path finding)

Aspect	Simple explanation
What it is	Finds a working network path between two devices, even when both are behind routers and firewalls.
Everyday analogy	A GPS that tries several routes; if every road is closed, it uses a ferry (the TURN relay).
Where it sits	Inside the connectivity layer; uses servers in the Network layer.

How it works, step by step:

- Gathers candidates: local address, public address (from STUN), and relay address (from TURN).
- Exchanges candidates with the other side through signaling.
- Runs connectivity checks on every candidate pair at the same time.
- Selects the best working pair (direct is preferred, relay is the fallback).
- Keeps checking the chosen path and switches if it fails.

Typical problems found here:

- Calls failing on strict corporate or hotel Wi-Fi where only TURN over TCP/TLS works.
- Timeouts too short on slow cellular networks.
- Switching from Wi-Fi to cellular not triggering a new path search.

How it was debugged and fixed:

- Packet captures (rvictl on iPhone, Wireshark on Mac) showed STUN requests with no responses, proving UDP was blocked.
- Fixes: add TURN over TCP/TLS fallback, tune timeouts, restart ICE on network change.

Transport (RTP / SRTP over UDP)

Aspect	Simple explanation
What it is	Splits compressed audio/video into encrypted packets, sends them, and receives packets from the other side.

Aspect	Simple explanation
Everyday analogy	A postal service that puts the movie into numbered, sealed envelopes and delivers them very fast.
Where it sits	Between the Encoder/Jitter Buffer and the network.

How it works, step by step:

- Packetizes compressed frames into RTP packets (each with a sequence number and timestamp).
- Encrypts them with SRTP so nobody can listen in.
- Sends them over UDP using the path chosen by ICE.
- Receives incoming packets, decrypts them, and passes them to the Jitter Buffer.
- Sends and receives RTCP reports (loss, delay) used to estimate bandwidth.

Typical problems found here:

- Packets larger than the network allows (MTU) being silently dropped, especially on VPNs.
- Decryption failures after a key refresh, causing sudden silence.
- Socket closed while still sending, causing crashes.

How it was debugged and fixed:

- Packet captures compared packet sizes with the path MTU and showed the big ones never arrived.
- Fixes: smaller packet size limit, correct key-rollover handling, safe socket lifetime.

Network Monitor and Reconnect Logic

Aspect	Simple explanation
What it is	Watches the device's network (Wi-Fi, cellular, none) and recovers the session when it changes or drops.
Everyday analogy	A car's satnav that instantly recalculates the route when you miss a turn.
Where it sits	Inside the connectivity layer, feeding Transport and the Session Manager.

How it works, step by step:

- Uses NWPathMonitor to detect network changes.
- Measures packet loss and round-trip time continuously.
- On a change or drop: tells the Session Manager to enter Reconnecting.

- Restarts ICE to find a new path while keeping the session alive.
- Uses exponential backoff (waits a little longer between each retry) and gives up after a limit.

Typical problems found here:

- Reconnect storms: many retries at once, overwhelming the server.
- Session marked “failed” during a brief tunnel or elevator signal loss that would have recovered.

How it was debugged and fixed:

- Trace logs with timestamps showed retry timing and how long outages really lasted.
- Fixes: backoff with random jitter, a short grace period before declaring failure, seamless ICE restart.

2.5 Network and Servers

Component	Simple explanation
Signaling server	A matchmaker. It helps the two devices find each other and exchange setup messages. It does not carry the audio/video.
STUN server	A mirror. It tells a device what its public internet address looks like from outside.
TURN / media server	A relay. When a direct path is blocked, audio/video is sent through this server instead.
Remote device	The other participant’s phone or computer, running the same SDK, doing the mirror-image of everything above.

2.6 Observability and Debugging Toolkit (added in this project)

These are the tools that made the 60+ root-cause fixes possible. Without them, engineers were guessing; with them, every failure left clear evidence.

Trace Logging (os_log and signposts)

Aspect	Simple explanation
What it is	Structured, time-stamped messages written by every layer, plus signposts that mark the start and end of important operations.
Everyday analogy	An aeroplane’s flight recorder (black box).

Aspect	Simple explanation
Where it sits	Built into every component; read with Console.app, log collect, or Instruments.

How it works, step by step:

- Each session gets a unique session ID that is printed on every log line, so one call can be followed across all layers.
- Logs use categories (media, sdk, connectivity) and levels (debug, info, error, fault).
- Signposts mark intervals such as “ICE gathering” or “first video frame” so their duration can be measured.
- Sensitive data (tokens, user IDs) is marked private so it is hidden in production logs.
- Logs can be collected from test devices or from users who opt in to share diagnostics.

Typical problems found here:

- Before: logs were sparse, unstructured and had no session ID, so different calls were mixed together.

How it was debugged and fixed:

- After: any failed session can be reconstructed step by step, showing exactly which layer failed first.

Packet Capture (tcpdump, rvtctl, Wireshark)

Aspect	Simple explanation
What it is	A recording of all network traffic during a session.
Everyday analogy	CCTV footage of the post office showing every envelope that came in and went out.
Where it sits	Taken on the Mac (tcpdump), on an iPhone via a Remote Virtual Interface (rvtctl), or on a network router.

How it works, step by step:

- Start a capture before reproducing the issue (e.g. `rvtctl -s <device-id>`, then `tcpdump` on the virtual interface).
- Reproduce the failure.
- Open the .pcap file in Wireshark.
- Filter for STUN, TURN, RTP, RTCP or the signaling connection.

- Check whether packets were sent, received, delayed, lost, reordered or oversized.

Typical problems found here:

- Answers the key question: is the network at fault, or is our code at fault?

How it was debugged and fixed:

- Example: if RTP packets arrive in the capture but the logs show the jitter buffer dropped them, the bug is in our code, not the network.

Crash Reports and Symbolication (dSYM, atos)

Aspect	Simple explanation
What it is	Turning a crash report full of hexadecimal memory addresses into readable function names, files and line numbers.
Everyday analogy	Translating a message written in code numbers into plain English using a codebook (the dSYM).
Where it sits	Crash reports come from TestFlight, Xcode Organizer or a crash-reporting service; dSYMs are saved at build time.

How it works, step by step:

- Archive the dSYM file for every build (matched by its unique build UUID).
- Collect the crash report (.ips) from the device or crash service.
- Match the crash report's UUID to the right dSYM.
- Symbolicate with Xcode or atos to convert addresses into function names and line numbers.
- Group crashes by their top symbolicated frames to find the most common ones.

Typical problems found here:

- Before: crashes in the media framework and SDK showed only addresses, so they could not be traced.

How it was debugged and fixed:

- After: each crash points to the exact line, e.g. a decoder callback using freed memory, which can then be reproduced and fixed.

LLDB (live debugger)

Aspect	Simple explanation
What it is	A tool that pauses the running app to look inside it.
Everyday analogy	Pausing a film to examine one frame in detail.
Where it sits	Attached through Xcode or the command line to a test device or simulator.

How it works, step by step:

- Set breakpoints (stop at this line) and conditional breakpoints (stop only if the state is wrong).
- Set watchpoints (stop whenever this variable changes) to catch unexpected state changes.
- Inspect variables, threads and call stacks with commands such as bt, frame variable and thread list.
- Use Address Sanitizer and Thread Sanitizer alongside LLDB to catch memory and threading bugs.

Typical problems found here:

- Used for bugs that are reproducible but whose cause is unclear from logs alone.

How it was debugged and fixed:

- Example: a watchpoint on the session state showed a background thread setting Connected after the user had already ended the call.

Instruments (CPU, memory, leaks, timing)

Aspect	Simple explanation
What it is	Apple's performance-measurement tool that records what the app does over time.
Everyday analogy	A fitness tracker for the app: heart rate (CPU), weight (memory), and a timeline of activity.
Where it sits	Run from Xcode against a device.

How it works, step by step:

- Time Profiler: shows which functions use the most CPU.
- Allocations and Leaks: show memory growth and objects that are never freed.
- System Trace and Thread States: show threads blocking or waiting on each other.
- os_signpost / Points of Interest: displays the trace-log signposts on the same timeline.

- Network instrument: shows connections and data volume.

Typical problems found here:

- Used for slow-burn issues: overheating, battery drain, memory growth during long calls, stutter.

How it was debugged and fixed:

- Example: Allocations showed encoder buffers piling up during long calls until iOS killed the app for using too much memory.

Reliability Metrics (99.9% session success)

Aspect	Simple explanation
What it is	The scoreboard that measures how many sessions succeed.
Everyday analogy	A school report card that shows whether grades are actually improving.
Where it sits	Collected from session-end events and logs, shown on dashboards.

How it works, step by step:

- Every session reports an outcome: success, failed-to-connect, dropped, or crashed.
- Session success rate = $\text{successful sessions} \div \text{total sessions} \times 100$.
- Failures are grouped by layer and error code, so the biggest problems are fixed first.
- After each fix is released, the rate is watched to confirm the improvement and catch regressions.

Typical problems found here:

- Without metrics, there is no proof that fixes worked.

How it was debugged and fixed:

- Tracking showed the success rate rising release by release until it reached 99.9%.

3. Workflow Explained (end to end)

There are two workflows to understand. The first is how a normal session flows through the system. The second is the debugging workflow used to find and fix each of the 60+ issues.

3.1 Runtime workflow: life of one session

The letters (A, B, C ...) match the arrows on the architecture diagram in Section 4.

Step	What happens (simple terms)	Evidence the toolkit records
1. Start (A)	The user taps “Start”. The App UI calls the SDK’s public API.	Log: API call received with session ID.
2. Validate (B)	The public API checks inputs and permissions, then hands the request to the Session Manager.	Log: validation result.
3. Enter Connecting (E, F)	The Session Manager changes state to Connecting. It tells the media framework to start capture (E) and the connectivity layer to start signaling (F).	Signpost begins: “session setup”.
4. Signaling (O)	Signaling opens a secure WebSocket to the server and exchanges the offer/answer with the remote device.	Packet capture shows TLS and WebSocket traffic.
5. Path finding (L, M)	ICE gathers candidates using STUN/TURN, tests them, and picks the best path for Transport.	Signpost: “ICE gathering” and “ICE connected”.
6. Capture and encode (G)	Camera and mic produce raw frames; the encoder compresses them.	Log: encoder started, resolution, bitrate.
7. Send (H, P)	Transport packetizes, encrypts (SRTP), and sends packets to the remote device.	Packet capture shows RTP/SRTP flowing out.
8. Receive (P, I)	Packets from the remote device are decrypted and placed in the jitter buffer.	Packet capture shows RTP flowing in.
9. Decode and play (J, K)	The jitter buffer releases frames to the decoder, which passes pictures and sound to the renderer.	Signpost: “first frame rendered”.
10. Connected (C, D)	The Session Manager moves to Connected; a callback tells the app, which updates the screen.	Log: state change with time-to-connect.

Step	What happens (simple terms)	Evidence the toolkit records
11. Keep alive (N)	During the call, the network monitor watches quality; bitrate adjusts automatically.	Periodic stats: loss, delay, bitrate.
12. Network change	If Wi-Fi drops, state moves to Reconnecting, ICE restarts on cellular, and media resumes.	Log: reconnect reason and duration.
13. End	The user taps “End”. The Session Manager tears down in order: stop capture → stop encoder → close transport → send “bye” → release decoder/renderer → Ended.	Log: outcome reported to metrics.

3.2 Debugging workflow: how each of the 60+ issues was solved

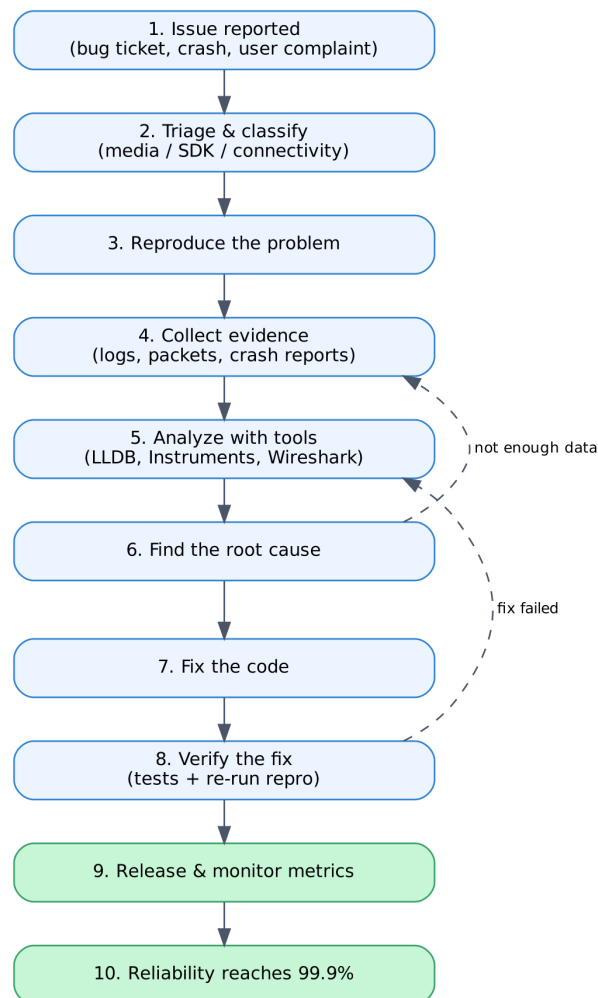


Figure 1. The repeatable debugging workflow used for every issue.

Step	What happens (simple terms)	Detail / tools used
1. Issue reported	A problem arrives from a bug ticket, QA, a crash report, a customer complaint or a dip in the metrics dashboard.	Write down symptoms, device, iOS version, network type, app/SDK version and time.
2. Triage and classify	Decide which layer is most likely involved (media, SDK or connectivity) and how serious it is (how many users, crash or annoyance).	Prioritize by impact: crashes and connection failures first.
3. Reproduce	Make the problem happen again on purpose, on a test device.	Use a network link conditioner to simulate bad networks, switch Wi-Fi/cellular, receive phone calls, run long calls.
4. Collect evidence	Gather all the clues from the same moment: trace logs, packet capture, crash report, and Instruments recording.	Session ID ties logs, packets and metrics together.
5. Analyze	Read the evidence like a detective. Find the first thing that went wrong, not the last.	Wireshark for packets, symbolicated stacks for crashes, LLDB for state, Instruments for performance.
6. Find the root cause	Ask “why?” repeatedly until the true cause is found (the “5 Whys”). If evidence is missing, add more logging and go back to step 4.	Example chain: video froze → decoder had no keyframe → keyframe request lost → request not retried → missing retry logic.
7. Fix	Change the code to remove the root cause, not just hide the symptom.	Keep the fix small and add a log line that would have caught it.
8. Verify	Re-run the reproduction steps and automated tests. Confirm the issue is gone and nothing else broke. If it failed, return to step 5.	Add a regression test so it never comes back.
9. Release and monitor	Ship the fix and watch the metrics dashboard for that failure type.	Compare failure rate before and after.

Step	What happens (simple terms)	Detail / tools used
10. Reliability improves	Each fix removes a slice of failures; together they raised session success to 99.9%.	Document the fix and lessons learned.

3.3 Example issues walked through end to end

Symptom	Layer	How the root cause was found	Fix
One-way audio after a phone call interruption	Media (Capture)	Trace logs showed the audio interruption “began” event but no “ended” handling; the mic never restarted.	Handle the interruption-ended event and restart the audio unit.
Calls never connect on hotel Wi-Fi	Connectivity (ICE/TURN)	Packet capture showed STUN requests over UDP with no reply: UDP was blocked.	Added TURN over TLS on port 443 as a fallback.
Random crash at end of call	SDK + Media (Decoder)	Symbolicated crash pointed to a decoder callback after release; LLDB confirmed teardown order was wrong.	Fixed teardown order and protected decoder lifetime.
Video freezes after about 30 minutes	Media (Jitter Buffer)	Packets arrived (capture) but were discarded (logs) right when sequence numbers wrapped past 65,535.	Corrected wrap-around comparison.
App killed during long calls	Media (Encoder)	Instruments Allocations showed encoded buffers never freed.	Released buffers in the encoder callback.
Session stuck on “Connecting...”	SDK (State Machine)	Signposts showed ICE never completed and no timeout fired.	Added per-state timeouts with clear error codes.

4. Architecture Diagram

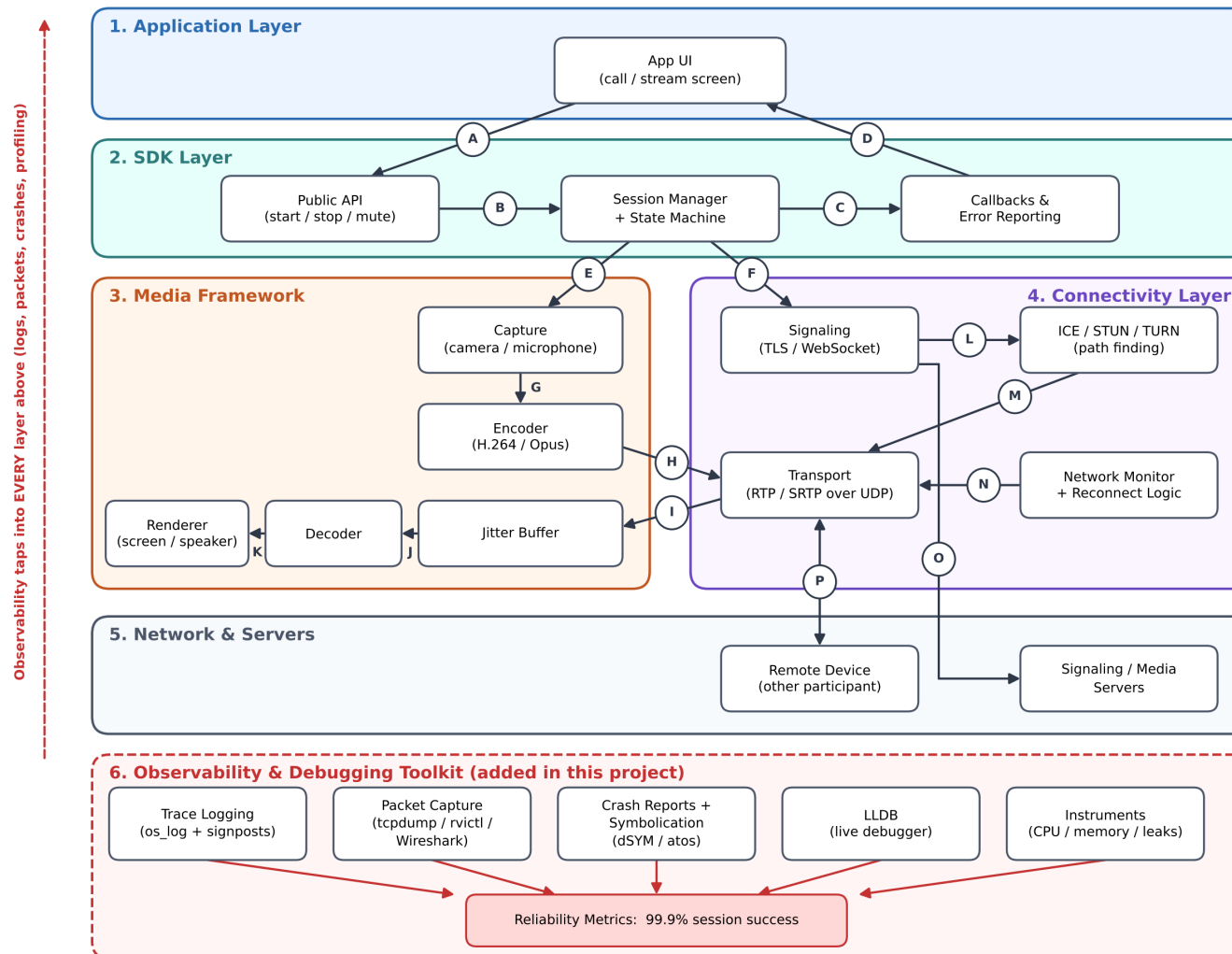


Figure 2. End-to-end architecture. Solid arrows = main data/control flow (letters A–P). Red dashed area = observability toolkit added by this project.

4.1 How to read the diagram

Visual element	Meaning
Coloured rounded rectangles (bands)	Each band is one layer of the system, numbered 1 to 6 from top to bottom.
White boxes	Individual components inside a layer.
Solid dark arrows with a letter	The main flow of requests, events or data. Letters match the runtime workflow in Section 3.1.
Two-headed arrow (P)	Data flows both ways: packets are sent to and received from the remote device.
Red dashed band (6)	The observability and debugging toolkit added in this project.
Red dashed arrow on the left	Shows that the toolkit watches every layer above it.
Red solid arrows inside band 6	Each tool's findings feed into the reliability metrics.
Pink box	The final outcome: 99.9% session success.

4.2 Layer-by-layer explanation

Layer	Colour	Components	Role in simple terms
1. Application	Blue	App UI	What the user sees and taps.
2. SDK	Teal	Public API, Session Manager + State Machine, Callbacks & Error Reporting	The toolkit that runs the session and reports progress to the app.
3. Media Framework	Orange	Capture, Encoder, Jitter Buffer, Decoder, Renderer	Turns real-world sound and pictures into data and back again.
4. Connectivity	Purple	Signaling, ICE/STUN/TURN, Transport, Network Monitor + Reconnect	Finds a network path and moves data securely across it.
5. Network & Servers	Grey	Remote Device, Signaling/Media Servers	Everything outside the user's phone.

Layer	Colour	Components	Role in simple terms
6. Observability	Red (dashed)	Trace Logging, Packet Capture, Crash Symbolication, LLDB, Instruments, Metrics	The clues and tools used to find and fix problems, plus the scoreboard.

4.3 Arrow-by-arrow explanation

Arrow	From → To	What travels along it	Simple explanation
A	App UI → Public API	Commands	The app asks the SDK to start, stop or mute.
B	Public API → Session Manager	Validated request	The checked request is handed to the brain of the SDK.
C	Session Manager → Callbacks	State events	The brain announces changes like “connected” or “reconnecting”.
D	Callbacks → App UI	Events and errors	The app is told what happened so it can update the screen.
E	Session Manager → Capture	Start/stop media	The brain tells the camera and mic to begin or end.
F	Session Manager → Signaling	Start/stop connection	The brain tells the network layer to set up or close the call.
G	Capture → Encoder	Raw frames and audio	Recorded pictures and sound go to be compressed.
H	Encoder → Transport	Compressed media	Compressed data goes to be packed into network envelopes.
I	Transport → Jitter Buffer	Incoming packets	Received envelopes go to the waiting room to be put in order.
J	Jitter Buffer → Decoder	Ordered frames	Frames are released steadily to be uncompressed.
K	Decoder → Renderer	Pictures and sound	Uncompressed media goes to the screen and speaker.

Arrow	From → To	What travels along it	Simple explanation
L	Signaling → ICE/STUN/TURN	Network candidates	Possible addresses are exchanged so path finding can begin.
M	ICE/STUN/TURN → Transport	Chosen path	The best working route is given to Transport to use.
N	Network Monitor → Transport	Network changes, quality	Transport is told when the network changes so it can recover.
O	Signaling → Servers	Setup messages (TLS)	The setup conversation goes through the signaling server.
P	Transport ↔ Remote Device	Encrypted RTP/SRTP packets	Actual audio and video travel both ways.

4.4 Observability connections (red elements)

Tool	Which layers it watches	What it reveals
Trace Logging	All layers (1–4)	The order and timing of every important event in a session.
Packet Capture	Connectivity (4) and Network (5)	Whether packets were really sent, received, lost, delayed or blocked.
Crash Reports + Symbolication	SDK (2), Media (3), Connectivity (4)	The exact function and line where a crash happened.
LLDB	Any layer, on a test device	Live variable values, thread states and who changed what.
Instruments	Any layer, on a test device	CPU, memory, leaks, thread blocking and timing.
Reliability Metrics	Whole system	The proof: session success climbing to 99.9%.

4.5 Full end-to-end trace through the diagram

1. The user taps Start in the **App UI** (blue). Arrow **A** carries the command down to the **Public API** (teal).

2. Arrow **B** passes it to the **Session Manager**, which switches the state machine to Connecting.
3. Arrow **F** tells **Signaling** (purple) to connect; arrow **O** carries the setup messages to the **servers** (grey).
4. Arrow **L** gives network candidates to **ICE/STUN/TURN**, which tests routes and, via arrow **M**, gives the winner to **Transport**.
5. At the same time arrow **E** starts **Capture** (orange). Arrow **G** feeds the **Encoder**, and arrow **H** sends compressed media to **Transport**.
6. Arrow **P** carries encrypted packets to and from the **Remote Device**.
7. Incoming packets travel along arrow **I** to the **Jitter Buffer**, then **J** to the **Decoder**, then **K** to the **Renderer**, and the user sees and hears the other person.
8. The Session Manager now announces Connected via arrow **C**, and arrow **D** tells the App UI to show the call screen.
9. Throughout, arrow **N** lets the **Network Monitor** steer Transport through network changes.
10. Throughout, the red **observability toolkit** records logs, packets, crashes and performance from every layer, and feeds the **metrics** that show 99.9% session success.

5. Results and Impact

Area	Before	After
Visibility	Sparse, mixed-up logs; unreadable crash reports; no network evidence.	Session-ID trace logs, signposts, packet captures and symbolicated crashes for every failure.
Time to root cause	Days of guessing; many issues closed as “cannot reproduce”.	Hours: evidence points to the failing layer and line directly.
Issues resolved	Recurring, unexplained failures.	60+ complex issues fixed at the root across media, SDK and connectivity.
Session reliability	Noticeable share of calls dropping, freezing or crashing.	99.9% of sessions succeed end to end.
Prevention	Same bugs returning.	Regression tests and better logging stop old bugs coming back.

5.1 How to explain this in an interview (30 seconds)

Talking points

“Our calling/streaming sessions were failing in ways we couldn’t explain. I built the observability we were missing: structured trace logs with a session ID across every layer, packet captures to see what the network was really doing, and crash symbolication so every crash pointed to a line of code. Using that evidence with LLDB and Instruments, I root-caused and fixed more than 60 issues across the media pipeline, the SDK state machine and the connectivity layer, which took session reliability to 99.9%.”